

An automated test data generation method for void pointers and function pointers in C/C++ libraries and embedded projects

Lam Nguyen Tung, Hoang-Viet Tran, Khoi Nguyen Le, Pham Ngoc Hung *

University of Engineering and Technology, Vietnam National University Hanoi, E3 building, 144 Xuan Thuy, Cau Giay, Hanoi, Viet Nam

ARTICLE INFO

Keywords:

Automated test data generation
Concolic testing
Symbolic execution
Void pointer
Function pointer
VFP verification

ABSTRACT

Automated test data generation for unit testing C/C++ functions using concolic methods is well-known for improving software quality while reducing human testing effort. However, there have been only a few researches related to generating test data for void pointers and function pointers which are commonly used in C/C++ libraries and embedded projects. This paper proposes a concolic-based method named VFP (Void and Function Pointers test data generation) to generate test data for void pointers and function pointers. The key idea of VFP method is to preprocess the source code of the project under test to find all possible types of void pointers and references of function pointers. These types and references are used in the initial test data generating phase of the concolic testing method. VFP method is implemented in VFP verification tool to test on various C/C++ libraries and embedded projects. The experimental results show that VFP significantly improves the coverage of the generated test data in comparison with existing methods.

1. Introduction

The popularity of pointers in C/C++ projects, in practice, attracts a lot of attention as many researches have been published in regards to this data type [1–4]. To the best of our knowledge, not only general pointers but also void and function pointers are commonly used in practice. As the result, quality assurance for C/C++ projects, in general, are of importance, especially for projects which use void and function pointers. One of the well-known quality assurance methods is automated software testing which is widely applied in both research and industrial communities. The automated testing includes two main approaches called static testing [5–8] and dynamic testing [9,10]. The static testing focuses on the internal structure of the project to generate test data without executing the project in a runtime environment. Although this approach can be used to test safety critical software systems and other industrial systems, the approach cannot be fully automated due to the high cost [5,6,8]. In the dynamic testing approach, the physical behavior of software systems is examined. The software must actually be compiled and run with given input values from test data generated either manually or automatically. Concolic testing [11–21] (known as dynamic symbolic execution [13] or concolic execution [21]) is a combination of static testing and dynamic testing which can employ advantages from both testing approaches. Concolic testing focuses on generating test data by executing projects under test. This method only partially generates test data at random. The remaining test data is generated based on the initial test data

and the unit's control flow graph corresponding to the unit under test. Based on the randomly generated initial test data, some branches and statements in the given function are covered. Then, path selection strategies explore all uncovered branches and statements to achieve maximum coverage as soon as possible. Due to this approach, concolic testing easily reaches higher coverage in comparison with static testing [20,22,23]. This approach has been widely applied to various industrial projects to increase test coverage and detect bugs with the modest human effort. Concolic testing attracts much attention from both research and industrial communities resulting in many research papers [11–20,22,23] and tools [13,21,24–29].

Although many researches and tools on concolic testing have been published, there are only a few researches related to generating test data for void pointers and function pointers. Some methods attempt to randomly construct a suitable initial test data in a specified range for a number of times until the initial test data does not cause any runtime errors such as DART [13], PathCrawler [28], SMART [30], CREST [31], CAUT [32]. Recently, instead of executing the source code with manually or randomly constructed inputs, other methods apply concolic testing to execute the source code with symbolic inputs. The symbolic inputs are initially allowed to be “anything” that can crash real code such as KLEE [25], EXE [26], SAVIOR [33]. However, it is noticed that concolic testing is incapable of generating test data for void pointers and function pointers in C/C++ libraries and embedded

* Corresponding author.

E-mail addresses: tunglam@vnu.edu.vn (L.N. Tung), thv@vnu.edu.vn (H.-V. Tran), khoi.n.le@vnu.edu.vn (K.N. Le), hungpn@vnu.edu.vn (P.N. Hung).

<https://doi.org/10.1016/j.infsof.2022.106821>

Received 17 April 2021; Received in revised form 6 November 2021; Accepted 2 January 2022

Available online 19 January 2022

0950-5849/© 2022 Elsevier B.V. All rights reserved.

projects. Void pointers are used to indirectly access objects with unknown types via their addresses. The type of a pointer is derived from the type of the referenced object. This is to ensure that the size and type of the referenced object could be determined. To the best of our knowledge, there has not been any proposal to handle void pointers. In addition to void pointers, function pointers are used to simplify code to dynamically select a function to execute. Although, function pointers are important in many projects, there are only MAIST method supporting function pointers [24]. However, MAIST method could not cover branches whose conditions depend on a function pointed by a function pointer. For example, MAIST method could not generate various execution scenarios enforced by assigning different functions to the function pointer. MAIST method attempts to find a set of candidate function references by statically examining all target source files and identifying functions that are assigned to the function pointer [24]. For this reason, when there are few functions assigned to a function pointer, it is difficult to generate test data, especially in library source codes testing.

In order to address the above issues, this paper proposes a method named Void and Function Pointers test data generation (VFP) to generate test data for void pointers and function pointers. Although VFP method focuses on C/C++ libraries and embedded projects, it can be applied to more general projects. The key idea of the proposed method is to analyze the given source code to find all possible types for void pointers and references for function pointers. The analysis is performed on source code both inside and outside of the unit under test. These types and references are used to generate the initial test data for concolic testing method. The generation process starts by running the unit under test with the initial test data to obtain a list of covered test paths. From these test paths, VFP method finds an uncovered node and the shortest uncovered test path from the root of the unit's control flow graph to that uncovered node using Dijkstra algorithm [34]. Then, the new test data to cover this shortest uncovered test path is generated and passed to the unit to find covered test paths. The process is repeated until no uncovered test path can be found. The main difference between VFP method and other concolic testing methods is that VFP method can find more initial test data by preprocessing the source code of the unit and its project. The reason is that existing methods do not support generating test data for void pointers which are set to *NULL*. For function pointers, VFP method searches for matching functions in the whole project and in the function itself. For this reason, VFP method could find more initial test data than MAIST method. With this larger initial test data, the generated test data set could cover many test scenarios with real test data instead of *NULL* as in MAIST method. This enables VFP method to obtain a significantly higher coverage than that of MAIST method, especially with units using void and/or function pointers. VFP method shows a potential applicability for projects in practice.

The rest of this paper is organized as follows. Section 2 describes the overview of VFP method with the key concept. Section 3 presents the details of VFP method. Section 4 discusses VFP verification tool to demonstrate VFP method and the experimental results. Section 5 reviews related works for the test data generation for C/C++ projects topic in detail. Section 6 gives the conclusion of the paper.

2. The overview of VFP method

The key idea of VFP method is based on the concolic testing method. VFP method introduces a new concept of type and reference maps. These maps are used in the preprocessing phase of VFP method to identify possible types and references of the given void pointer and function pointer parameters. These maps are used to generate initial test data for the concolic testing method.

2.1. Concolic testing

Concolic testing, which is the combination of the *Concrete* and *Symbolic* execution, was first mentioned by Larson and Austin [11] in 2003. The name "Concolic" was proposed by Sen et al. [21] in 2005. The overview of concolic testing method is shown in Fig. 1.

In concolic testing method, there are three steps as follows:

1. Initial test data set is generated at random.
2. Execute test data generated from step 1 or step 3 and analyze covered test paths.
3. If an uncovered test path exists, the method generates a test data to cover the test path. Otherwise, the testing process stops.

It is noticed that initial test data generation at step 1 greatly affects the whole testing process. The reason is that initial test data set could cover some of the test paths, then affects directly to the number of uncovered test paths in step 3. VFP method bases on this observation to generate test data for void and function pointers.

2.2. Type map and reference map

2.2.1. Type map

Type map is defined to handle void pointers. A type map is to store the types of void pointers. Type map is a data structure, for which its key contains the name of a parameter while its value is a list of all possible types. VFP method uses a type map to choose a type for each void pointer. For example, let consider the type map `{{data, {int, double, complex_number}}}` corresponding to the source code listed in Code 1. In this type map, `data` is a key of a void pointer parameter which could be mapped to three possible types `int`, `double`, or `complex_number`. Details of the construction of a type map with the corresponding unit under test are shown in Section 3.1.

Code 1: Example code for void pointer preprocessing

```
#include <stdio.h>
typedef struct complex_number
{
    int real;
    int imagination;
};
void computeAndDisplay(void* data, int opt) {
    switch(opt) {
        case 0: {
            int* int_ptr = (int*) data;
            printf("Integer number: %d", *int_ptr);
            break;
        }
        case 1: {
            double* double_ptr = (double*) data;
            printf("Double number: %.2lf", *double_ptr);
            break;
        }
        case -1: {
            complex_number* cn_ptr = (complex_number*) data;
            complex_number cn = *cn_ptr;
            printf("Complex number: %d + %di", cn.real,
                cn.imagination);
            break;
        }
    }
}
```

2.2.2. Reference map

Reference map is defined to handle function pointers. A reference map is to store the references of function pointers. Reference map is a data structure, for which its key contains the name of a parameter while its value is a list of all possible references. VFP method uses a reference map to choose a reference for each function pointer. For example, let consider the reference map `{{hash_func, {int_hash(void*), pointer_hash(void*), string_hash(void*), string_nocase_hash(void*)}}}` corresponding to the source code listed in Code 2. In this reference

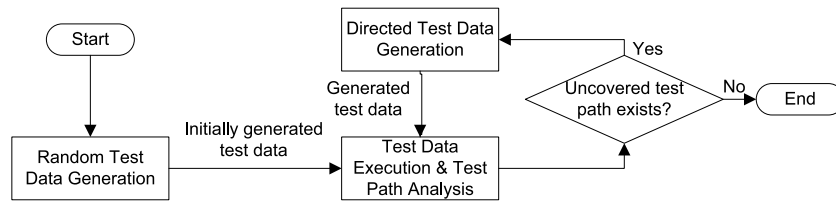


Fig. 1. Overview of concolic testing method.

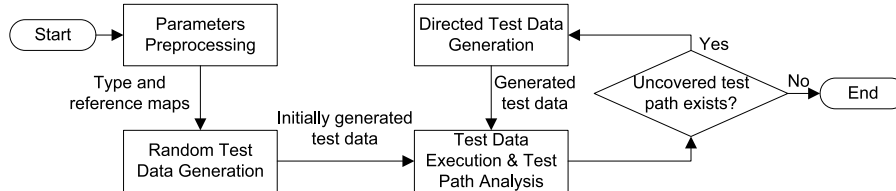


Fig. 2. Test data generation for void pointers & function pointers.

map, hash_func is a key of a function pointer parameter which is mapped to four possible function references `int_hash(void*)`, `pointer_hash(void*)`, `string_hash(void*)`, or `string_nocase_hash(void*)`. Details of the construction of a reference map with the corresponding unit under test are shown in Section 3.1.

Code 2: Example code for function pointer preprocessing

```

typedef unsigned int (*BloomFilterHashFunc)(void* data);
BloomFilter*bloom_filter_new(unsigned int table_size,
                             BloomFilterHashFunc hash_func) {
    /* logic code */
}
unsigned int int_hash(void *location);
unsigned int pointer_hash(void *location);
unsigned int string_hash(void *str);
unsigned int string_nocase_hash(void *str);
  
```

2.3. VFP overview

VFP method follows the approach of concolic testing [13,21] by combining concrete testing and symbolic testing. Fig. 2 describes VFP method containing four main phases.

In the first phase, **Parameters Preprocessing** is an additional process to the original concolic testing method. **Parameters Preprocessing** is to handle void pointers and function pointers. It finds all suitable types for void pointers and references for function pointers. This is performed in source code both inside and outside of the unit under test. In the second phase, **Random Test Data Generation** creates combinations of these types and references to generate test data randomly. In the third phase, **Test Data Execution and Test Paths Analysis** executes all generated test data to obtain their covered test paths. Then, the method attempts to explore an uncovered test path. If an uncovered test path exists, **Directed Test Data Generation**, in the fourth phase, converts this newly explored test path to path constraints by using the symbolic execution method [35]. The generated path constraints, in forms of logic expressions, are transformed into an SMT-Lib expression as an input to be solved by an SMT-Solver. The solution of this SMT-Solver, newly generated test data, is fed back into **Test Data Execution and Test Paths Analysis** to further explore an uncovered test path. This process is repeated until there is not any uncovered test path. The method returns the generated test data set.

Algorithm 1 VFP algorithm

Input: f - unit under test; P - project under test.

Output: T : generated test data set

```

1:  $type\_map \leftarrow$  call Algorithm 2 ( $f$ )           ▷ Parameters Internal Preprocessing
2:  $type\_map, ref\_map \leftarrow$  call Algorithm 3 ( $f, P$ )   ▷ Parameters External Preprocessing
3:  $T \leftarrow$  call Algorithm 4 ( $f, type\_map, ref\_map$ )   ▷ Random Test Data Generation
4: while true do
5:    $uncovered\_test\_path \leftarrow$  call Algorithm 5 ( $f, T$ )   ▷ Test Data Execution and Test Paths Analysis
6:   if  $uncovered\_test\_path = NULL$  then
7:     break
8:   else
9:      $t \leftarrow$  call Algorithm 6 ( $uncovered\_test\_path$ )   ▷ Directed Test Data Generation
10:     $T \leftarrow T \cup t$ 
11:  end if
12: end while
13: return  $T$ 
  
```

2.4. VFP algorithm

Algorithm 1, VFP algorithm, describes the proposed concolic-based method to generate test data for void pointers and function pointers. In the first phase, **Parameters Preprocessing** (line 1 – 2) finds all suitable types ($type_map$) for void pointers and references (ref_map) for function pointers. This phase is performed on source code inside of the unit under test (**Parameters Internal Preprocessing** phase, line 1), and then outside of the unit under test (**Parameters External Preprocessing** phase, line 2). In the second phase, **Random Test Data Generation** generates test data at random (line 3). This randomly generated test data is the input to the repeated process to explore uncovered test paths (line 4 – 12). During this process, the third phase, **Test Data Execution and Test Paths Analysis** executes generated test data to determine uncovered test paths (line 5). This process terminates when there is not any uncovered test paths (line 6 – 7). Otherwise, in the fourth phase, **Directed Test Data Generation** generates test data directly (line 9) and appends it to the test data set (line 10). VFP algorithm returns the test data set T (line 13).

3. Details of VFP method

3.1. Parameters preprocessing

This phase obtains type maps and reference maps of the unit under test by performing two subroutines, [Parameters Internal Preprocessing](#) to analyze the source code of the unit under test, and then, [Parameters External Preprocessing](#) to analyze the functions calling the unit under test and the whole project under test. All possible types for void pointers and references for function pointers are evaluated to construct instances of type maps and reference maps.

3.1.1. Parameters internal preprocessing

The key idea of this subroutine, Algorithm 2, is to check all void pointers' usages to find all possible types (through cast expressions and assignments). It also checks if each function pointer has any dereferences. Our experiments show that the unit under test without function pointer dereferences could have maximum coverage with a *NULL* reference. [Parameters Internal Preprocessing](#) accepts the unit under test *f* and a type map *type_map* as input. Note that the "parameter" term is used to call the variable defined in the corresponding function prototype and its attributes or elements.

Algorithm 2 Parameters Internal Preprocessing

Input: *f* - unit under test;
Output: *type_map* - type map corresponding to *f*;
1: **for each** *expression* **in** *f* **do**
2: **if** *expression* **is a** cast expression **or an** assignment **then**
3: *name, type* ← **call** Extract-parameter-name-and-type(*expression*)
4: *type_map* ← *type_map* ∪ {(*name, type*)}
5: **else if** *expression* **is a** function call expression **then**
6: **if** *expression* called by a function pointer *parameter* **then**
7: **call** Mark-function-pointer-as-dereferenced(*parameter*)
8: **else if** *expression*'s arguments contains void or function pointers **then**
9: *f'* ← **call** Extract-function-being-called(*expression*)
10: *type_map* ← *type_map* ∪ **call** Algorithm 2 (*f'*)
11: **end if**
12: **end if**
13: **end for**
14: **return** *type_map*

The method traverses every *expression* node in the abstract syntax tree of the unit under test *f* (line 1). Each *expression* is classified as one of three following cases. In the first case (line 2–4), *expression* is a cast expression, e.g., `int *location = (int *) void_ptr` or an assignment, e.g., `int *cast_ptr = void_ptr`. The corresponding *name* and *type* of the parameter are extracted from *expression* and stored in *type_map*. In the second case (line 5–7), *expression* is a function call expression by dereferencing a function pointer. In this case, the dereferenced function pointer is marked. In the third case (line 8–10), *expression* is a function call expression whose arguments contain void pointers or function pointers. The algorithm searches the function *f'* called in the corresponding *expression* (line 9) by recursively calling Algorithm 2 with parameters of the function *f'* (line 10). This process terminates when all expressions in the unit under test *f* and every called function *f'* are visited.

3.1.2. Parameters external preprocessing

This subroutine, Algorithm 3, contains two main tasks in which the first task handles all function calls corresponding to the unit under test, and the second task handles the whole project under test. In the first task, for each function call, the algorithm filters all the passing arguments corresponding to void pointers and function pointers, which are the potential types and references. For void pointer arguments, the algorithm extracts the types of passing arguments and stores in *type_map*. For function pointer arguments marked as *dereferenced*, the algorithm identifies the function references of passing arguments and

Algorithm 3 Parameters External Preprocessing

Input: *f* - unit under test; *P* - project under test;
Output: *type_map* - type map corresponding to *f*; *ref_map* - reference map corresponding to *f*;
1: Get a set of function call expressions *f_caller* corresponding to *f*
2: **for each** *call_expression* **in** *f_caller* **do**
3: **for each** *argument* **in** *call_expression* **do**
4: **if** *argument* **is a** void pointer **then**
5: *name, type* ← **call** Extract-name-and-type(*argument, call_expression*)
6: *type_map* ← *type_map* ∪ {(*name, type*)}
7: **else if** *argument* **is a** function pointer **then**
8: **if** corresponding parameter is marked as *dereferenced* **then**
9: *name, type* ← **call** Extract-name-and-function-reference(*argument*)
10: *ref_map* ← *ref_map* ∪ {(*name, reference*)}
11: **end if**
12: **end if**
13: **end for**
14: **end for**
15: **for each** *parameter* **in** *f* **do**
16: **if** *parameter* **is a** function pointer **then**
17: *param_matches* ← **call** Filter-all-functions-matching-signature(*f*)
18: **for each** *function* **in** *param_matches* **do**
19: *name* ← **call** Get-name(*parameter*)
20: *ref_map* ← *ref_map* ∪ {(*name, function*)}
21: **end for**
22: **end if**
23: **end for**

stores in *ref_map*. In the second task, the algorithm searches the whole project under test to find all functions matched the function pointer signature.

Details of this subroutine is shown in Algorithm 3. The first task (line 1–14) starts by getting a set of function call expressions named *f_caller* corresponding to the unit under test *f*. Then, for each passing *argument* (line 3) of each function call expression in *f_caller* (line 2), if *argument* is a void pointer (line 4), the corresponding *name* and *type* of *argument* are extracted and stored in *type_map* (line 5–6). If *argument* is a function pointer marked as *dereferenced*, the corresponding *name* and function *reference* of the function pointer *argument* are extracted and stored in *ref_map* (line 7–12). The second task (line 15–23) is to filter all function pointer *parameters* of the unit under test *f*. For each function pointer *parameter* (line 16), the algorithm searches the whole project to find all matched functions *param_matches* which have the same parameters and return type (line 17) and stores in *ref_map* (line 18–21).

3.1.3. Parameters preprocessing examples

Code 3 shows the source code of `int_equal` function which has two void pointer parameters `vptr1` and `vptr2`. For this source code, Section 3.1.1 constructs this type map {(`vptr1`, {`int`}), (`vptr2`, {`int`})}. Then, in Section 3.1.2, a double pointer and a character pointer are passed as arguments. So after this step, the type map is updated as {(`vptr1`, {`int`, `double`}), (`vptr2`, {`int`, `char`})}.

Code 3: Example code for void pointer preprocessing

```
int int_equal(void *vptr1, void *vptr2) {
    int *ptr1 = (int *) vptr1;
    int *ptr2 = (int *) vptr2;
    return *ptr1 == *ptr2;
}

int main() {
    double f = 5; double *f_ptr = &f;
    char c = 'a'; char *c_ptr = &c;
    return int_equal(f_ptr, c_ptr);
}
```

Code 4 shows the source code of `bloom_filter_new` function which has a function pointer parameter `hash_func`. [Parameters External Preprocessing](#) returns this reference map `{(hash_func, {int_hash(void*), pointer_hash(void*), string_hash(void*), string_nocase_hash(void*)})}`. The reference map has four references (`string_hash(void*)` is a direct passing argument and the others are matching functions).

Code 4: Example code for function pointer preprocessing

```
typedef unsigned int (*BloomFilterHashFunc)(void* data);

BloomFilter *bloom_filter_new(unsigned int table_size,
                             BloomFilterHashFunc hash_func,
                             unsigned int num_functions);

unsigned int int_hash(void *ptr);
unsigned int pointer_hash(void *ptr);
unsigned int string_hash(void *str);
unsigned int string_nocase_hash(void *str);

int main() {
    BloomFilterHashFunc func = &string_hash;
    BloomFilter *filter = bloom_filter_new(4, func, 4);
    return 0;
}
```

3.2. Random test data generation

The key idea of this phase is to generate some initial test data by using type maps and reference maps for void pointers and function pointers, respectively and random values generation for other data types.

Algorithm 4 Generating initial test data

Input: f - unit under test; $type_map$ - type map generated from Phase 1; ref_map - reference map generated from Step 1.

Output: T : collection of generated test data

```
1:  $T \leftarrow \emptyset$ 
2:  $combinations \leftarrow \{(type, ref) \mid type \in type\_map, ref \in ref\_map\}$ 
3: for each combination in combinations do
4:    $test\_data \leftarrow \emptyset$ 
5:   for each parameter in  $f$  do
6:      $param\_value \leftarrow NULL$ 
7:     if parameter is a void pointer then
8:        $type \leftarrow \text{call Get-real-type}(parameter, combination)$ 
9:       if type is  $NULL$  then
10:         $type \leftarrow \text{default type}$ 
11:       end if
12:        $param\_value \leftarrow \text{call Generate-random-value}(parameter, type)$ 
13:     else if parameter is a function pointer then
14:        $param\_value \leftarrow \text{call Get-function-reference}(parameter, combination)$ 
15:       if  $param\_value$  is  $NULL$  then
16:         $param\_value \leftarrow \text{default reference}$ 
17:       end if
18:     else
19:        $param\_value \leftarrow \text{call Generate-random-value}(parameter)$ 
20:     end if
21:      $test\_data \leftarrow test\_data \cup \{(parameter, param\_value)\}$ 
22:   end for
23:  $T \leftarrow T \cup \{test\_data\}$ 
24: end for
25: return  $T$ 
```

From the type maps and reference maps generated by [Parameters Preprocessing](#), combinations are created from types and references in these maps. Then, for each combination, test data is randomly generated with types and references chosen from the combination and added to the test data set. Although combining all available types and references does not create all good test data, this does not affect the coverage result of the generated test data set. The reason is that, in [Test Data Execution and Test Paths Analysis](#), with a not good test data,

the execution of the function being tested will throw an exception or can cover only a few statements. This does not affect coverage results of other test data.

The method to randomly generate test data is described in Algorithm 4 whose inputs are the unit under test f , a type map $type_map$, and a reference map ref_map . Firstly, the test data set, denoted by T , is initialized as an empty set (line 1). Then, $combinations$ are generated from types in $type_map$ and references in ref_map (line 2). For each combination (line 3), $test_data$ is initialized as an empty set (line 4). After that, with each $parameter$ of the unit under test f (line 5), its value, $param_value$, is initialized as $NULL$ (line 6). Each $parameter$ is classified as one of three following cases to generate $param_value$. In the first case (line 7–12), $parameter$ is a void pointer (line 7), its type, denoted by $type$, is retrieved from $combination$ (line 8). If this $type$ does not exist, it is assigned the default type, type `int` in C/C++ (line 9–11). Then, $param_value$ is generated randomly by using the found $type$'s values range (line 12). In the second case (line 13–17), $parameter$ is a function pointer. The function reference $param_value$ is either retrieved from $combination$, or assigned a default function which is an arbitrary function matching the function pointer (line 14–17). In the third case (line 19), if $parameter$ is neither a void pointer nor a function pointer, the algorithm will generate a random value using its corresponding values range and assigns it to $param_value$. After $param_value$ is generated, the pair of $(parameter, param_value)$ is appended to $test_data$ (line 21). Finally, the generated test data is appended to the test data set T (line 23). This generated test data set T is returned (line 25).

3.3. Test data execution and test paths analysis

All newly generated test data, from [Random Test Data Generation](#), are executed and their covered test paths are collected from the corresponding execution. Then, this phase attempts to find an uncovered node and a shortest uncovered test path from the root to that node. In this path selection process, the feasibility of the selected path has already been checked.

Details of this phase are described in Algorithm 5. The algorithm executes all new test data in T (line 1). Then, all covered test paths are collected from the corresponding execution of test data (line 2).

The algorithm finds an uncovered node ($uncovered_node$) from the

Algorithm 5 Finding an Uncovered Test Path

Input: f : unit under test; T : collection of newly generated test data;

Output: $uncovered_test_path$: new explored test path

```
1: call Execute-all-new-test-data( $T$ )
2:  $test\_paths \leftarrow \text{call Get-all-covered-test-paths}()$ 
3:  $uncovered\_node \leftarrow \text{call Find-an-uncovered-node}(test\_paths)$ 
4:  $uncovered\_test\_path \leftarrow \text{call Find-the-shortest-path-from-root}(uncovered\_node)$ 
5: return  $uncovered\_test\_path$ 
```

collection of covered test paths ($test_paths$) (line 3) and the shortest uncovered test path ($uncovered_test_path$) from the unit's control flow graph root to $uncovered_node$ (line 4–5). There are many algorithms which can be used to find this shortest path such as Depth-First Search, Breadth-First Search, Dijkstra, A*, etc. VFP verification tool, shown in Section 4, employs Dijkstra algorithm. The reason is that this is the best algorithm to find shortest path between the two given vertices in a directed graph with nonnegative edge weight.

3.4. Directed test data generation

The main purpose of this phase is to generate a test data to cover the uncovered test path found in [Test Data Execution and Test Paths Analysis](#). From a given uncovered test path, path constraints are generated. These generated path constraints are transformed into an SMT-Lib expression as an input to be solved by an SMT-Solver. Based on the solution of this SMT-Solver, a new test data is generated.

Algorithm 6 Generating Directed Test Data

Input: *uncovered_test_path*: an uncovered test path
Output: *t*: next test data

```

1: t ← NULL
2: path_constraints ← call Symbolic-execution(uncovered_test_path)
3: solution ← call Solve(path_constraints)
4: if solution is not NULL then
5:   t ← call Generate-test-data(solution)
6: end if
7: return t

```

Details of this phase are presented in Algorithm 6. The algorithm takes an uncovered test path explored from **Test Data Execution and Test Paths Analysis** called *uncovered_test_path* as its input, and the output is the corresponding test data *t*. The algorithm begins with *t* initialized as *NULL* (line 1). Then, *uncovered_test_path* is converted to corresponding *path_constraints* by using symbolic execution (line 2). Next, *path_constraints*, in forms of logic expressions, are transformed into an SMT-Lib expression as an input to be solved by an SMT-Solver. Then, an SMT-Solver is employed to solve the SMT-Lib expressions to obtain the corresponding *solution* (line 3). If *solution* exists (line 4), *t* is created from *solution* (line 5). Finally, the algorithm returns *t* (line 7). This newly generated test data *t* is fed back into **Test Data Execution and Test Paths Analysis** to further explore another uncovered test path.

4. Implemented tool and experiments

In order to evaluate the effectiveness of VFP method, experiments are performed to highlight three key points as follows: (i) the ability to generate test data for void pointers; (ii) the comparison between VFP method and MAIST method, which was developed by Kim et al. [24], in term of generating test data for function pointers; and (iii) the comparison between VFP method and MAIST method in term of generating test data for other data types. The effectiveness of VFP method is evaluated through its corresponding coverage, running time and memory usage when generating test data for each unit under test. In our experiments, we use the statement coverage [36] (at function level) to evaluate coverage in all experiments (i.e., *statement coverage* = *number of covered statements* / *total number of statements of a function*). We implemented VFP method, MAIST method, and the traditional concolic testing method in a tool named VFP verification¹ using Java, IntelliJ IDEA Ultimate 2020.2 IDE. Experiments are performed on a machine with the following specifications: Ubuntu 18.04, Intel® Core™ i5-3470 CPU @ 3.20 GHz × 4, 8 GBs RAM memory.

4.1. Experimental benchmarks

Experiments are performed on four open source libraries and embedded projects repositories.

- **C-Algorithms**² (17780 LOC - Lines of code): C is a programming language which includes very limited standard libraries in comparison to other modern programming languages such as Java, Python, Javascript, etc. This is a collection of common Computer Science algorithms.
- **Libusbhost**³ (1059 LOC): Libusbhost is similar to Libusb. While Libusb is one of the most popular USB user space drivers on Linux, Libusbhost is adapted to Android USB host requirements. The main role of this library is to act as an interface between

Table 1

Experimental results for void pointers of C-Algorithms.

File	Units	↑	Traditional Method				VFP Method			
			Cov	Num	Mem	Time	Cov	Num	Mem	Time
compare-int.c	2/2	1	77.8	16	260	101	100	12	334	147
trie.c	13/14	2	76.3	219	391	252	85.7	161	886	582
binary-heap.c	5/6	2	85.1	94	541	173	97.4	160	593	546
avl-tree.c	23/24	3	80.1	360	877	1875	84.0	325	1034	1307
bloom-filter.c	7/8	3	65.5	130	522	171	80.0	108	711	854
set.c	15/16	5	70.3	252	1031	521	81.2	219	950	1218
sortedarray.c	10/11	0	76.7	177	639	163	85.2	142	804	1023
compare-string.c	4/4	2	68.1	32	759	91	100	4	1226	963

Linux kernel USB driver and Android USB framework. This makes it possible to implement necessary infrastructure to facilitate detection of any new USB device connected to the kernel.

- **AGL Framework Binder**⁴ (50766 LOC): This project provides the binder component of the microservice architecture of Automotive Grade Linux (AGL).
- **Libuv**⁵ (101121 LOC): Libuv is a multi-platform support library with a focus on asynchronous I/O. It supports epoll, kqueue, Windows IOCP, and Solaris event ports. It was originally an abstraction around Libev or Microsoft IOCP because Libev only supports the select action but not the poll action and IOCP on Windows.

4.2. The effectiveness of generating test data for void pointer parameters

In order to measure the effectiveness of VFP method in generating test data for void pointers, VFP method is compared to the traditional concolic testing method in some key aspects including statement coverage, memory usage and running time using VFP verification tool. For given testing repositories, functions, which have void pointer parameters, are filtered and test data are generated for them using both methods. From these test data, we obtain the corresponding coverages for each method. Tables 1–4 show experimental results corresponding to project C-Algorithms, AGL Framework Binder, Libusbhost, and Libuv. The labels for each column in Tables 1–4 are as follows:

- Column “File”: the file name under consideration.
- Column “Units”: the number of units under test per number of all functions in the file.
- Column “Cov”: the achieved statement coverage (in %) at function level of all filtered units under test.
- Column “↑”: the number of units under test achieving more than 10% (i.e., coverage of VFP method - coverage of the traditional concolic testing method) the coverage of the traditional concolic testing method.
- Column “Num”: the minimum number of test data generated to achieve that coverage.
- Column “Mem”: the average memory usages (in MB) when generating each test data for each unit.
- Column “Time”: the average running time (in millisecond) required for generating each test data for each unit.

From these results, we have the following observations:

- Experimental results of C-Algorithms and AGL Framework Binder projects show that even if there is not any function call expression of the unit under test, VFP method still can generate test data based on casting expressions in the given unit and achieve a coverage of 78.9%.

¹ <https://gitlab.com/tungtohi/vfpv-tool>.

² <https://github.com/fraggle/c-algorithms>.

³ <https://android.googlesource.com/platform/system/core/+/master/libusbhost/>.

⁴ <https://gerrit.automotivelinux.org/gerrit/gitweb?p=src/app-framework-binder.git>.

⁵ <https://github.com/libuv/libuv/tree/v1.x/src>.

Table 2

Experimental results for void pointers of AGL Framework Binder.

File	Units	↑	Traditional Method				VFP Method			
			Cov	Num	Mem	Time	Cov	Num	Mem	Time
websock.c	10/29	0	80.4	103	728	9838	80.4	80	839	11949
jobs.c	2/30	1	57.4	33	515	6781	82.4	33	530	7756
afb-apiset.c	9/45	3	60.0	204	238	638	77.2	202	224	1300
afs-discover.c	1/1	0	27.6	16	336	3778	27.6	16	364	4948
afb-proto-ws.c	10/80	1	43.3	151	402	2099	49.0	146	456	5106
afb-monitor.c	2/20	0	75.0	12	613	731	75.0	12	1066	882

Table 3

Experimental results for void pointers of Libusbhost.

File	Units	↑	Traditional Method				VFP Method			
			Cov	Num	Mem	Time	Cov	Num	Mem	Time
usbhost.c	10/46	1	73.2	158	534	239	76.6	131	409	162

Table 4

Experimental results for void pointers of Libuv.

File	Units	↑	Traditional Method				VFP Method			
			Cov	Num	Mem	Time	Cov	Num	Mem	Time
locks	2/3	2	8.7	32	229	139	100	2	204	67
thread-create	2/3	2	16.7	30	342	39	100	2	81	57

- With Libusbhost project, instead of dereferences, void pointers are passed in library functions or assigned another pointer. VFP method generates default values for these cases and achieves a coverage of 76.6%.
- Libuv project does not have any dereferences of void pointers and their usages are simple so VFP method achieves a coverage of 100%.
- In all projects, the coverage of the test data set generated by VFP method is greater than that of the traditional concolic testing method.
- There are 7/8 files of C-Algorithms, 3/6 files of AGL Framework Binder, 1/1 files of Libusbhost, and 2/2 files of Libuv projects for which VFP method can increase by more than 10% of coverage for some of the function in those files in comparison to the traditional concolic testing method. In our opinion, each percentage of the coverage, which we can increase by improving our test data set, contributes to the quality of the given project. As the result, in this case, 10% coverage improved by VFP method generated test data set is significant.
- The minimum number of test data generated by VFP method is smaller than that of the traditional concolic testing method (except for *binary-heap.c* file of C-Algorithms). This is a good result as it is always good to minimize the number of test data while increasing its corresponding coverage.
- In term of memory usage, for 7/8 files of C-Algorithms, 1/6 files of AGL Framework Binder, 1/1 files of Libusbhost, and 2/2 files of Libuv projects, VFP method takes less memory usage than the traditional concolic testing method when generating test data. There is no obvious reason for this difference between the memory usage of the two methods.
- In term of running time, for 4/8 files of C-Algorithms, 3/6 files of AGL Framework Binder, 1/1 files of Libusbhost, and 2/2 files of Libuv projects, VFP method takes less time than the traditional concolic testing method when generating test data. There is no obvious proof for the difference between the time required to generate data of the two methods.
- The memory usage and running time of both methods depends on the size and the structure of projects under test. For this reason, for different projects, we can see different memory and time required to generate data for both methods.

Table 5

Experimental results for C-Algorithms.

File	Units	MAIST method				VFP method			
		Cov	Num	Mem	Time	Cov	Num	Mem	Time
avl-tree.c	6	80.3	50	254	541	80.3	59	250	2299
binary-heap.c	6	83.8	55	169	373	94.0	53	206	375
bloom-filter.c	8	60.0	88	373	274	78.9	71	316	591
set.c	16	71.6	204	445	1488	82.8	183	383	1638
sortedarray.c	11	68.5	109	553	2633	85.9	105	457	2231

Table 6

Experimental results for AGL Framework Binder.

File	Units	MAIST method				VFP method			
		Cov	Num	Mem	Time	Cov	Num	Mem	Time
afb-apiset.c	4	60.0	69	362	948	72.5	52	565	1130
afb-context.c	5	49.3	56	270	3547	49.3	62	419	3201
afb-proto-ws.c	4	56.5	39	612	3276	62.2	38	279	3247
afb-session.c	1	56.5	16	276	1750	62.5	16	468	2077
afb-xreq.c	10	100	14	269	1106	100	17	383	1760
afs-discover.c	1	27.6	16	436	2778	27.6	16	476	2596
jobs.c	8	80.8	72	432	698	84.5	101	285	869
sig-monitor.c	3	87.5	18	873	793	87.5	18	880	981
wrap-json.c	3	47.2	34	533	936	47.2	28	270	1095

4.3. Function pointer test data generations comparison

In order to compare VFP method and MAIST method with their test data generation effectiveness, experiments are performed on C-Algorithms and AGL Framework Binder projects shown in Section 4.2. We use key indicators such as the coverage (at function level), the minimum number of generated test data, the average memory usage needed, and the average running time required to generate test data for each unit under test. (see Figs. 3–8).

Tables 5 and 6 show the experimental results corresponding to C-Algorithms and AGL Framework Binder projects. The column labels in Tables 5 and 6 are the same as those of Tables 1–4. From the experimental results for function pointers, we have the following observations:

- VFP method is capable of generating test data for both void pointers and function pointers. VFP method effectively performs on units with pointer dereferences instead of simple pointer assignments or comparisons.
- In C-Algorithms project, the coverage of VFP method is higher than that of MAIST method. However, in the case of AGL Framework Binder project, its units under test do not have many pointer dereferences. For this reason, there is no significant difference between the two methods.
- The running time of VFP method is higher than that of MAIST method. [Parameters Preprocessing](#) is an additional phase of VFP method compared to traditional concolic testing method. This phase causes the main difference in memory usage and running time between the two methods. For this reason, the running time of VFP method depends on the size of the project under test. However, experimental results in Tables 5 and 6 show that the running time for VFP method is acceptable and applicable to real projects.

A Wilcoxon signed-rank test is performed with the coverage shown in Tables 5 and 6 to compare the effectiveness of the two methods. In the Wilcoxon signed-rank test result of AGL project, VFP method achieves higher coverage than MAIST method (medium 26.4%). In C-Algorithms project, the coverage achieved by VFP method is higher than that of MAIST method by 42.3%.

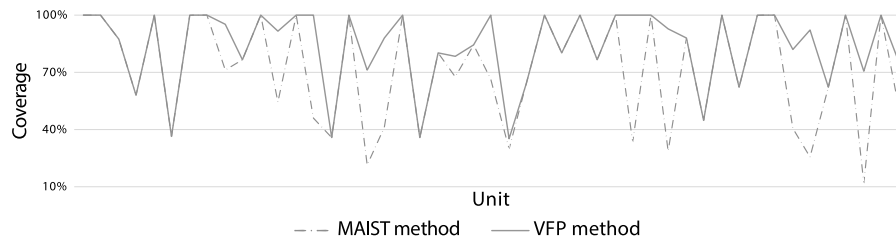


Fig. 3. Coverage comparasion of C-Algorithms.

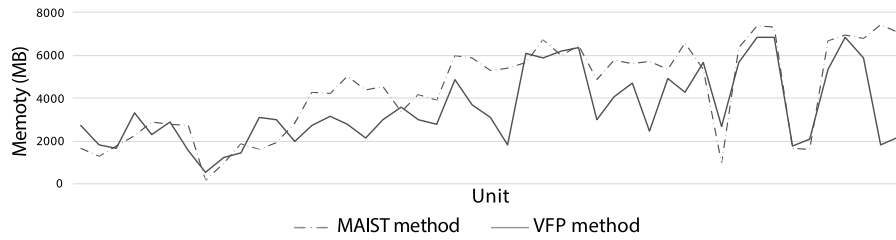


Fig. 4. Total Memory usages comparasion of C-Algorithms.

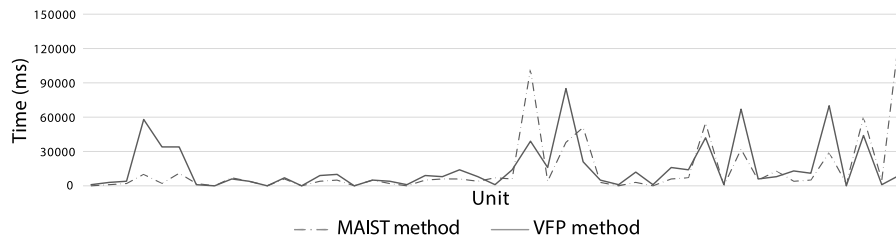


Fig. 5. Total Time comparasion of C-Algorithms.

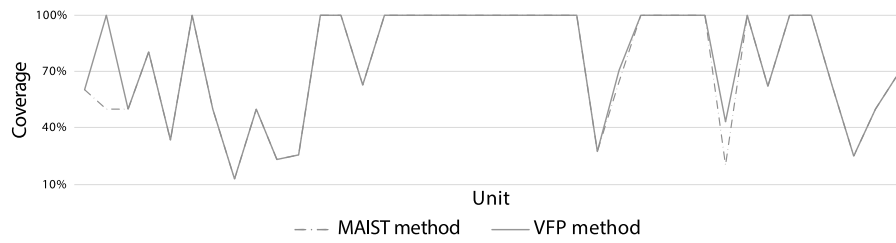


Fig. 6. Coverage comparasion of AGL Framework Binder.

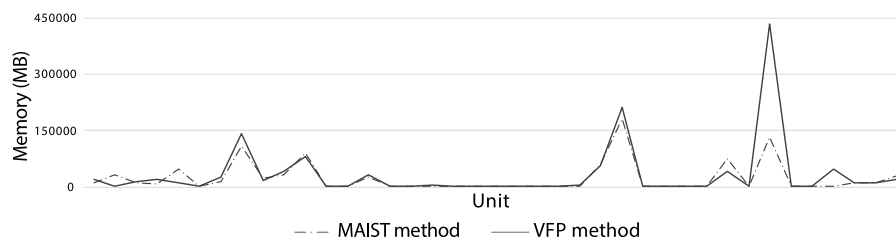


Fig. 7. Total Memory usages comparasion of AGL Framework Binder.

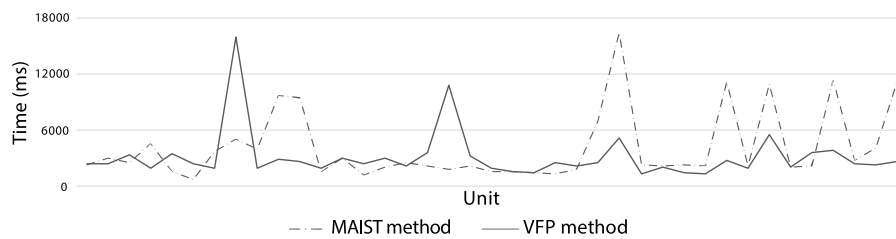


Fig. 8. Total Time comparasion of AGL Framework Binder.

Table 7
Experimental results for AGL Framework Binder.

File	Units	MAIST method				VFP method			
		Cov	Num	Mem	Time	Cov	Num	Mem	Time
fdev-epoll.c	5	87.0	36	671	1278	87.0	25	766	1306
afb-wsjl.c	31	73.7	231	785	2586	74.3	221	830	2822
subpath.c	3	61.1	24	919	529	61.1	24	1100	654
process-name.c	2	90.7	11	806	834	92.7	11	878	1155
afb-auth.c	8	68.1	64	333	1081	68.1	62	477	1417
afb-autoset.c	4	86.5	25	820	1463	86.5	23	842	1466
u16id.c	19	83.6	132	605	1127	84.7	142	670	1321
afb-token.c	6	84.6	27	735	1534	84.6	26	815	1564

Table 8
Experimental results for Libusbhost.

File	Units	MAIST method				VFP method			
		Cov	Num	Mem	Time	Cov	Num	Mem	Time
usbhost.c	36	85.0	159	486	1096	85.1	163	515	1163

4.4. Test data generations for other data types comparison

In order to compare VFP method and MAIST method with their test data generation effectiveness for other data types, experiments are performed with AGL Framework Binder and Libusbhost projects shown in Section 4.2. For given testing repositories, functions, which do not have void pointer parameters and function pointer parameters, are filtered and test data are generated for them using both methods. We use key indicators such as the coverage (at function level), the minimum number of generated test data, the average memory usage needed, and the average running time required to generate test data for each of the unit under test.

Tables 7 and 8 show the experimental results corresponding to AGL Framework Binder and Libusbhost projects. The column labels in Tables 7 and 8 are the same as those of Tables 1–4. From the experimental results for other data types, it is noticed that the running time and memory usage of VFP method are higher than that of MAIST method. However, experimental results in Tables 7 and 8 show that these differences are acceptable and applicable to real projects.

4.5. Discussions

From the experimental results, it can be seen that the coverage of both experiments are not 100% in all cases. There are several reasons for this circumstance. Most of them come from the fact that analyzing the source code of an arbitrary project is not easy. The most common scenarios are listed below with corresponding explanation.

- The unit uses `enum`, defined `const`: VFP verification tool has not supported these key words when analyzing the given unit.
- The unit calls other functions or units from the same project or from other libraries: This is the integration test and not in the scope of VFP method.
- The passing parameters are complex structures: VFP verification tool has not supported all types of structures yet.
- The unit contains *dead code*: Dead code is a part of the unit under test where the program can never run into. For this reason, dead code cannot be covered by any test data (e.g., `malloc() == NULL`).
- The method of symbolic execution has not supported all kinds of complex statements: This is the limitation of the implementation of the conversion of a test path to an SMT constraints.

Another observation from the experimental results is that the number of test data generated by VFP method is smaller whilst the coverage is higher than that of the existing methods due to the following reasons:

- When applying VFP method, if the coverage reaches 100%, VFP method will stop immediately. For this reason, the number of test data will be reduced. This is also the reason that in some cases, VFP method uses more memory and time than the traditional concolic method.
- VFP method checks if void pointers and function pointers had dereferences. If they do, VFP method will generate test data for them. Otherwise, VFP method sets *NULL* to those pointers. In contrast, existing methods assign *NULL* to void pointers by default. For function pointers, existing methods generate test data based on those pointers' assignment.

In regards to the validity of the experimental results, we think that it comes from both logical and practical aspects of VFP method. Logically, VFP method can generate test data for void pointers and generate test data set for function pointers. This is analyzed in Sections 2 and 3. Practically, for void pointers, we need to compare the effectiveness of VFP method with the traditional concolic testing method as no research has been carried out for void pointers. In addition, for function pointers, we implement both VFP method and MAIST method as we do not have tool supporting it. Experimental results shows that those results are corresponding to the analysis in logical aspect. Consequently, we think that this can ensure the reliability of VFP method.

5. Related works

Related to automated software testing, several works have been proposed. Concrete execution using random test data generation [37–41] is the simplest but still very effective. Some tools using bounded-exhaustive concrete execution [42–44] tries all values from domains provided by users. The code coverage gained by these tools is high, especially for testing the implementation of data structure. However, for retrieving such high code coverage, the user needs carefully choose values in the given domains.

Symbolic execution has a variety of approaches including abstraction-based model checking [45,46], explicit-state model checking [47], symbolic-sequence exploration [48], and static analysis [49]. Tools applying symbolic execution inherit the limitations of their underlying reasoning engines such as theorem provers and constraint solvers. For example, the incapability of analyzing source code that would build constraints out of pre-specified theories exists in tools using precise symbolic execution [47,48]. As another example, tools based on predicate abstraction [45,46] do not handle code that depends on complex data structures.

In 2005, Godefroid et al. [13] presented a new tool, named DART, for automatically testing software via incrementally generating test inputs by combining concrete and symbolic execution, which is called concolic testing afterward [21]. During testing, DART detects standard errors such as program crashes, assertion violations, and non-termination. While randomly generating test data, if a parameter's type is a pointer, its value is randomly initialized with either the value *NULL* or with the address of the newly allocated memory location whose length is corresponding to the type pointed by the parameter. A challenge in applying Godefroid et al.'s approach is to extract and solve the constraints generated by a program. This problem is particularly complex for programs which have dynamic data structures using pointer operations. In order to solve the above problem, Sen et al. [21] proposed CUTE whose key idea is to represent inputs for the unit under test using a logical input map that represents all inputs, including memory graphs, as a collection of scalar symbolic variables. Then, the logical input map is used to build constraints on these inputs by symbolically executing the code under test. However, these tools do not consider generating test data for void pointers and function pointers. Sharing the same concern with automated test data generation, especially for void pointers and function pointers, VFP method focuses on generating test data for them. VFP method can improve the achieved coverage and the quality of real projects in practice.

In 2008, Cadar et al. [26] presented a bug-finding tool named EXE and a symbolic execution tool named KLEE [25]. Both of them automatically generate inputs crashing real code, which checks if any possible value allowed by the current path condition would cause an error at each dangerous operation (e.g., pointer dereference). However, these tools do not support generating test data for function pointers. In addition, they can only handle void pointer parameters when their types are identified, which require manual investigation and huge human effort. In 2009, Godefroid et al. [27] presented a memory model for representing arbitrary symbolic pointer dereferences to memory regions accessible by a program during its execution. This memory model has been implemented in SAGE [50] which was presented in 2008. Like KLEE [25], SAGE does not support function pointers and requires manual identifying the types of void pointers to generate test data. To handle the above limitations, VFP method not only generates test data for function pointers but also supports identifying void pointer types automatically to generate test data using found types.

Having the same approach of building a new memory model, Zheng et al. [51] proposed a method to generate test data automatically for the unit test of C programs with derived types including pointers, which is implemented into a test data generation tool called CAUT. To handle pointers, this approach presents a Logical Memory Map which is maintained to map logical addresses to their values that are either logical addresses or primitive type values. However, this approach has a problem when it comes to generating test data for pointers. In CAUT, instead of handling deference operations directly, a pointer dereference is taken as a whole constraint and the handling is postponed in the test data generation phase. The above problem makes it difficult to generate test data for void pointers and function pointers. To resolve the above problem, VFP method combines types and references obtained from the preprocessing step with symbolic execution to dynamically generate test data for void pointers and function pointers.

In 2019, Yunho et al. [24] developed an automated testing method named MAIST that automatically generates test inputs for function pointers by applying concolic testing. The key idea of MAIST method is to statically examine all target source files and identify functions that are assigned to a function pointer and select a function in the list of candidate functions to generate test data. However, when there are few functions assigned to function pointers, it is difficult to generate test data, especially in library source codes testing. To solve the above problem, VFP method has an improvement which searches the whole project under test to find functions having the same signature as the function pointer signature. In the case of no matching function is found, a default function can be automatically generated and assigned to the function pointer.

From the above analysis, we can see that generating test data for pointers, especially for void and function pointers, attracts many attentions from both research and industrial communities. Although many researches and tools have been published, there are certain limitations. With VFP method, we want to contribute a feasible method to improve existing methods for a better test data set when testing projects in practice.

6. Conclusion

This paper presents VFP method to generate test data for void pointers and function pointers of C/C++ libraries and embedded projects, to which there are only a few researches related. The proposed method preprocesses the unit under test to find all possible types for void pointers and references for function pointers both inside and outside of the unit under test. Then, the concolic testing method begins with found types and references to generate required test data. In addition, this paper introduces VFP verification tool implementing VFP method. This tool is applied to test on various C/C++ large scale libraries and embedded projects in practice. To show the usefulness and effectiveness of VFP method, the experiments are performed on several projects. The

obtained results show that VFP method can generate a lower number of test data whilst reaching higher coverage in comparison with existing methods.

Although VFP method has achieved considerable results as aforementioned, there are still several remaining works need to be addressed. The method has not considered the case where various parameters have the same name. Moreover, when void pointers and function pointers are passed to a library call, this method has not been able to find suitable types or references due to the black-box nature of libraries. Furthermore, VFP verification tool needs to be applied to test on more complex and larger C/C++ projects in practice for a better evaluation of its performance. Last but not least, we will perform mutation testing to evaluate the quality of the generated test data set such as the ability to detect errors, size of test data set, etc.

CRedit authorship contribution statement

Lam Nguyen Tung: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft, Data curation. **Hoang-Viet Tran:** Methodology, Formal analysis, Investigation, Writing – review & editing, Supervision. **Khoi Nguyen Le:** Writing – review & editing. **Pham Ngoc Hung:** Main idea, Methodology, Formal analysis, Investigation, Writing – reviewing & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] S. Liu, G. Tan, T. Jaeger, PtrSplit: Supporting general pointers in automatic program partitioning, in: Proceedings Of The 2017 ACM SIGSAC Conference On Computer And Communications Security, in: CCS '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 2359–2371, <http://dx.doi.org/10.1145/3133956.3134066>.
- [2] M.J. Nam, P. Akritidis, D.J. Greaves, FRAMER: A tagged-pointer capability system with memory safety applications, in: Proceedings Of The 35th Annual Computer Security Applications Conference, in: ACSAC '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 612–626, <http://dx.doi.org/10.1145/3359789.3359799>.
- [3] V. Kuznetsov, L. Szekeres, M. Payer, G. Candea, R. Sekar, D. Song, Code-pointer integrity, in: The Continuing Arms Race: Code-Reuse Attacks And Defenses, Association for Computing Machinery and Morgan & Claypool, 2018, pp. 81–116.
- [4] S. Nagarakatte, J. Zhao, M.M. Martin, S. Zdancewic, SoftBound: Highly compatible and complete spatial memory safety for C, SIGPLAN Not. 44 (6) (2009) 245–258, <http://dx.doi.org/10.1145/1543135.1542504>.
- [5] G. Buckle, Static analysis of safety critical software (techniques, tools, and experiences), in: F. Redmill, T. Anderson (Eds.), Industrial Perspectives Of Safety-Critical Systems, Springer London, London, 1998, pp. 150–168.
- [6] B. Wichmann, A. Canning, D. Clutterbuck, L. Winsborrow, N. Ward, W. Marsh, Industrial perspective on static analysis, Softw. Eng. J. 10 (1995) 69–75, <http://dx.doi.org/10.1049/sej.1995.0010>.
- [7] B. Chess, J. West, Secure Programming With Static Analysis, first ed., Addison-Wesley Professional, 2007.
- [8] N. Ward, The static analysis of safety critical software using MALPAS, IFAC Proc. Vol. 22 (19) (1989) 91–96, [http://dx.doi.org/10.1016/S1474-6670\(17\)52801-3](http://dx.doi.org/10.1016/S1474-6670(17)52801-3), IFAC/IFIP Workshop on Safety of Computer Control Systems 1989 (SAFECOMP '89), Vienna, Austria, 5–7 December.
- [9] G.J. Myers, C. Sandler, The Art Of Software Testing, John Wiley & Sons, Inc. Hoboken, NJ, USA, 2004.
- [10] S. Anand, E.K. Burke, T.Y. Chen, J. Clark, M.B. Cohen, W. Grieskamp, M. Harman, M.J. Harrold, P. McMinn, An orchestrated survey of methodologies for automated software test case generation, J. Syst. Softw. 86 (8) (2013) 1978–2001, <http://dx.doi.org/10.1016/j.jss.2013.02.061>.
- [11] E. Larson, T. Austin, High coverage detection of input-related security faults, in: Proceedings Of The 12th Conference On USENIX Security Symposium - Vol. 12, USENIX Association, USA, 2003, p. 9.
- [12] K. Sen, Concolic testing, in: Proceedings Of The Twenty-Second IEEE/ACM International Conference On Automated Software Engineering, in: ASE '07, Association for Computing Machinery, New York, NY, USA, 2007, pp. 571–572, <http://dx.doi.org/10.1145/1321631.1321746>.

- [13] P. Godefroid, N. Klarlund, K. Sen, DART: Directed automated random testing, in: Proceedings Of The 2005 ACM SIGPLAN Conference On Programming Language Design And Implementation, in: PLDI '05, Association for Computing Machinery, New York, NY, USA, 2005, pp. 213–223, <http://dx.doi.org/10.1145/1065010.1065036>.
- [14] C. Cadar, D. Engler, Execution generated test cases: How to make systems code crash itself, in: P. Godefroid (Ed.), Model Checking Software, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 2–23.
- [15] R. Majumdar, K. Sen, Hybrid concolic testing, in: Proceedings Of The 29th International Conference On Software Engineering, in: ICSE '07, IEEE Computer Society, USA, 2007, pp. 416–426, <http://dx.doi.org/10.1109/ICSE.2007.41>.
- [16] K. Sen, Concolic testing and constraint satisfaction, in: K.A. Sakallah, L. Simon (Eds.), Theory And Applications Of Satisfiability Testing - SAT 2011, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 3–4.
- [17] X. Wang, J. Sun, Z. Chen, P. Zhang, J. Wang, Y. Lin, Towards optimal concolic testing, in: Proceedings Of The 40th International Conference On Software Engineering, in: ICSE '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 291–302, <http://dx.doi.org/10.1145/3180155.3180177>.
- [18] M. Kim, Y. Kim, G. Rothmel, A scalable distributed concolic testing approach: An empirical evaluation, in: Proceedings Of The 2012 IEEE Fifth International Conference On Software Testing, Verification And Validation, in: ICST '12, IEEE Computer Society, USA, 2012, pp. 340–349, <http://dx.doi.org/10.1109/ICST.2012.114>.
- [19] R. Kannavara, C.J. Havlicek, B. Chen, M.R. Tuttle, K. Cong, S. Ray, F. Xie, Challenges and opportunities with concolic testing, in: 2015 National Aerospace And Electronics Conference, NAECON, 2015, pp. 374–378, <http://dx.doi.org/10.1109/NAECON.2015.7443099>.
- [20] C. Cadar, K. Sen, Symbolic execution for software testing: Three decades later, Commun. ACM 56 (2) (2013) 82–90, <http://dx.doi.org/10.1145/2408776.2408795>.
- [21] K. Sen, D. Marinov, G. Agha, CUTE: A concolic unit testing engine for C, in: Proceedings Of The 10th European Software Engineering Conference Held Jointly With 13th ACM SIGSOFT International Symposium On Foundations Of Software Engineering, in: ESEC/FSE-13, Association for Computing Machinery, New York, NY, USA, 2005, pp. 263–272, <http://dx.doi.org/10.1145/1081706.1081750>.
- [22] D.-A. Nguyen, T.N. Huong, H.V. Dinh, P.N. Hung, Improvements of directed automated random testing in test data generation for C++ projects, Int. J. Softw. Eng. Knowl. Eng. 29 (09) (2019) 1279–1312, <http://dx.doi.org/10.1142/S0218194019500402>.
- [23] C.S. Păsăreanu, R. Kersten, K. Luckow, Q.-S. Phan, Chapter six - symbolic execution and recent applications to worst-case execution, load testing, and security analysis, in: A.M. Memon (Ed.), in: Advances in Computers, vol. 113, Elsevier, 2019, pp. 289–314, <http://dx.doi.org/10.1016/bs.adcom.2018.10.004>, URL <https://www.sciencedirect.com/science/article/pii/S0065245818300640>.
- [24] Y. Kim, D. Lee, J. Baek, M. Kim, Concolic testing for high test coverage and reduced human effort in automotive industry, in: 2019 IEEE/ACM 41st International Conference On Software Engineering: Software Engineering In Practice, ICSE-SEIP, 2019, pp. 151–160, <http://dx.doi.org/10.1109/ICSE-SEIP.2019.00024>.
- [25] C. Cadar, D. Dunbar, D. Engler, KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs, in: Proceedings Of The 8th USENIX Conference On Operating Systems Design And Implementation, in: OSDI'08, USENIX Association, USA, 2008, pp. 209–224.
- [26] C. Cadar, V. Ganesh, P.M. Pawlowski, D.L. Dill, D.R. Engler, EXE: Automatically generating inputs of death, ACM Trans. Inf. Syst. Secur. 12 (2) (2008) <http://dx.doi.org/10.1145/1455518.1455522>.
- [27] B. Elkarablieh, P. Godefroid, M.Y. Levin, Precise pointer reasoning for dynamic test generation, in: Proceedings Of The Eighteenth International Symposium On Software Testing And Analysis, in: ISSTA '09, Association for Computing Machinery, New York, NY, USA, 2009, pp. 129–140, <http://dx.doi.org/10.1145/1572272.1572288>.
- [28] N. Williams, B. Marre, P. Mouy, M. Roger, PathCrawler: Automatic generation of path tests by combining static and dynamic analysis, in: M. Dal Cin, M. Ka  n  che, A. Pataricza (Eds.), Dependable Computing - EDCC 5, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 281–292.
- [29] D.-A. Nguyen, T.N. Huong, H.V. Dinh, P.N. Hung, Improvements of directed automated random testing in test data generation for C++ projects, Int. J. Softw. Eng. Knowl. Eng. 29 (09) (2019) 1279–1312, <http://dx.doi.org/10.1142/S0218194019500402>.
- [30] P. Godefroid, Compositional dynamic test generation, SIGPLAN Not. 42 (1) (2007) 47–54, <http://dx.doi.org/10.1145/1190215.1190226>.
- [31] J. Burnim, K. Sen, Heuristics for scalable dynamic test generation, in: 2008 23rd IEEE/ACM International Conference On Automated Software Engineering, 2008, pp. 443–446, <http://dx.doi.org/10.1109/ASE.2008.69>.
- [32] T. Sun, Z. Wang, G. Pu, X. Yu, Z. Qiu, B. Gu, Towards scalable compositional test generation, in: 2009 Ninth International Conference On Quality Software, 2009, pp. 353–358, <http://dx.doi.org/10.1109/QSIC.2009.53>.
- [33] Y. Chen, P. Li, J. Xu, S. Guo, R. Zhou, Y. Zhang, T. Wei, L. Lu, SAVIOR: Towards bug-driven hybrid testing, in: 2020 IEEE Symposium On Security And Privacy, SP, 2020, pp. 1580–1596, <http://dx.doi.org/10.1109/SP40000.2020.00002>.
- [34] E. Dijkstra, A note on two problems in connexion with graphs, Numer. Math. 1 (1) (1959) 269–271.
- [35] J.C. King, Symbolic execution and program testing, Commun. ACM 19 (7) (1976) 385–394, <http://dx.doi.org/10.1145/360248.360252>.
- [36] L. Copeland, A Practitioner's Guide to Software Test Design, Artech House, Inc. USA, 2003.
- [37] D.L. Bird, C.U. Munoz, Automatic generation of random self-checking test cases, IBM Syst. J. 22 (3) (1983) 229–245, <http://dx.doi.org/10.1147/sj.223.0229>.
- [38] C. Csallner, Y. Smaragdakis, Jcrasher: An automatic robustness tester for java, Softw. Pract. Exp. 34 (11) (2004) 1025–1050, <http://dx.doi.org/10.1002/spe.602>.
- [39] J.E. Forrester, B.P. Miller, An empirical study of the robustness of windows NT applications using random testing, in: Proceedings Of The 4th Conference On USENIX Windows Systems Symposium - Vol. 4, in: WSS'00, USENIX Association, USA, 2000, p. 6.
- [40] A.J. Offutt, J.H. Hayes, A semantic model of program faults, SIGSOFT Softw. Eng. Notes 21 (3) (1996) 195–200, <http://dx.doi.org/10.1145/226295.226317>.
- [41] C. Pacheco, M.D. Ernst, Eclat: Automatic generation and classification of test inputs, in: A.P. Black (Ed.), ECOOP 2005 - Object-Oriented Programming, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 504–527.
- [42] C. Boyapati, S. Khurshid, D. Marinov, Korat: Automated testing based on java predicates, in: Proceedings Of The 2002 ACM SIGSOFT International Symposium On Software Testing And Analysis, in: ISSTA '02, Association for Computing Machinery, New York, NY, USA, 2002, pp. 123–133, <http://dx.doi.org/10.1145/566172.566191>.
- [43] W. Grieskamp, Y. Gurevich, W. Schulte, M. Veanes, Generating finite state machines from abstract state machines, in: Proceedings Of The 2002 ACM SIGSOFT International Symposium On Software Testing And Analysis, in: ISSTA '02, Association for Computing Machinery, New York, NY, USA, 2002, pp. 112–122, <http://dx.doi.org/10.1145/566172.566190>.
- [44] T. Xie, D. Marinov, D. Notkin, Rostra: A framework for detecting redundant object-oriented unit tests, in: Proceedings Of The 19th IEEE International Conference On Automated Software Engineering, in: ASE '04, IEEE Computer Society, USA, 2004, pp. 196–205.
- [45] T. Ball, Abstraction-Guided Test Generation: A Case Study, Tech. Rep. MSR-TR-2003-86, 2003, p. 14.
- [46] D. Beyer, A.J. Chlipala, T.A. Henzinger, R. Jhala, R. Majumdar, Generating tests from counterexamples, in: Proceedings Of The 26th International Conference On Software Engineering, in: ICSE '04, IEEE Computer Society, USA, 2004, pp. 326–335.
- [47] W. Visser, C.S. Pundefinedsundefinedreanu, S. Khurshid, Test input generation with java PathFinder, SIGSOFT Softw. Eng. Notes 29 (4) (2004) 97–107, <http://dx.doi.org/10.1145/1013886.1007526>.
- [48] T. Xie, D. Marinov, W. Schulte, D. Notkin, Symstra: A framework for generating object-oriented unit tests using symbolic execution, in: N. Halbwachs, L.D. Zuck (Eds.), Tools And Algorithms For The Construction And Analysis Of Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 365–381.
- [49] C. Csallner, Y. Smaragdakis, Check 'n' crash: Combining static checking and testing, in: Proceedings Of The 27th International Conference On Software Engineering, in: ICSE '05, Association for Computing Machinery, New York, NY, USA, 2005, pp. 422–431, <http://dx.doi.org/10.1145/1062455.1062533>.
- [50] P. Godefroid, M.Y. Levin, D. Molnar, Automated whitebox fuzz testing, in: Proceedings Of Network And Distributed Systems Security, NDSS 2008.
- [51] Z. Wang, X. Yu, T. Sun, G. Pu, Z. Ding, J. Hu, Test data generation for derived types in C program, in: 2009 Third IEEE International Symposium On Theoretical Aspects Of Software Engineering, 2009, pp. 155–162, <http://dx.doi.org/10.1109/TASE.2009.10>.